Habit Tracking App
Phase 2 – Implementation & Results

STUDENT: DOMINIK VOGEL

MATRICULATION NUMBER: 42201950

STUDY PROGRAM: ROBOTICS (B.ENG.)

OBJECT ORIENTED AND FUNCTIONAL PROGRAMMING (PYTHON)

IU INTERNATIONALE HOCHSCHULE

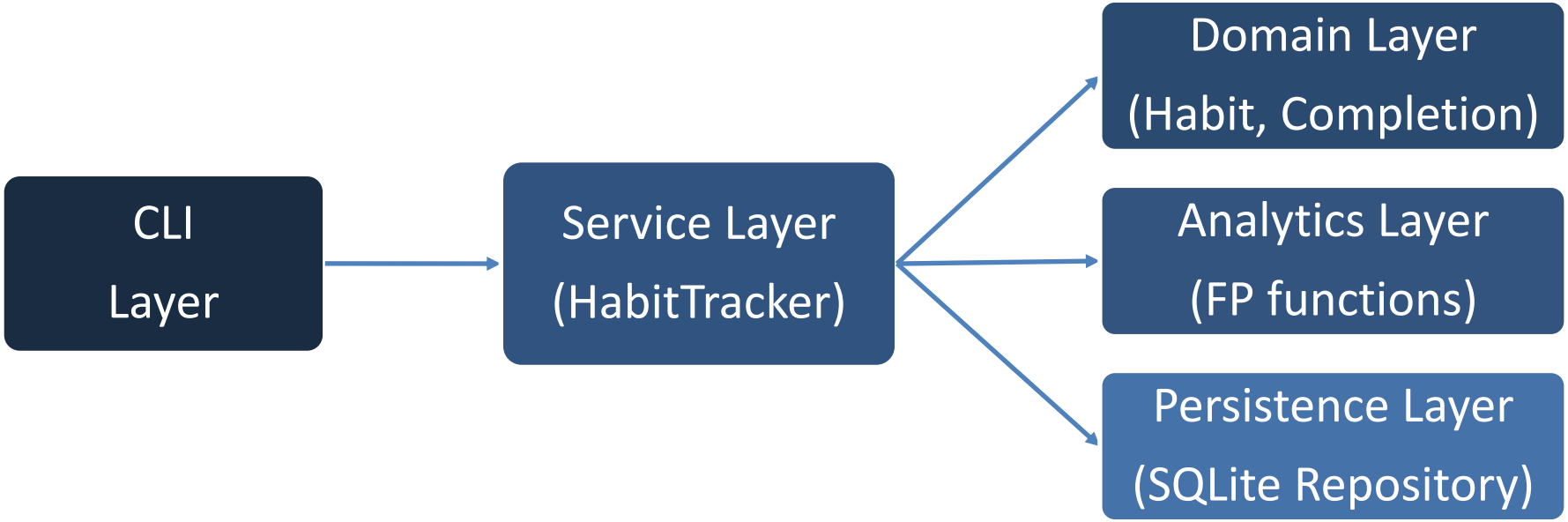
SUBMISSION DATE: 13.02.2026

PROJECT ASSIGNMENT PHASE 2

Project Overview & Goals

- CLI-based habit tracking application
- Daily and weekly habits
- Persistent data storage (SQLite)
- Streak analytics
- Focus on clean architecture, OOP & functional programming

Current Architecture



- CLI communicates exclusively with the HabitTracker service layer
- Service layer orchestrates domain, analytics and persistence
- Dependencies are strictly one-directional

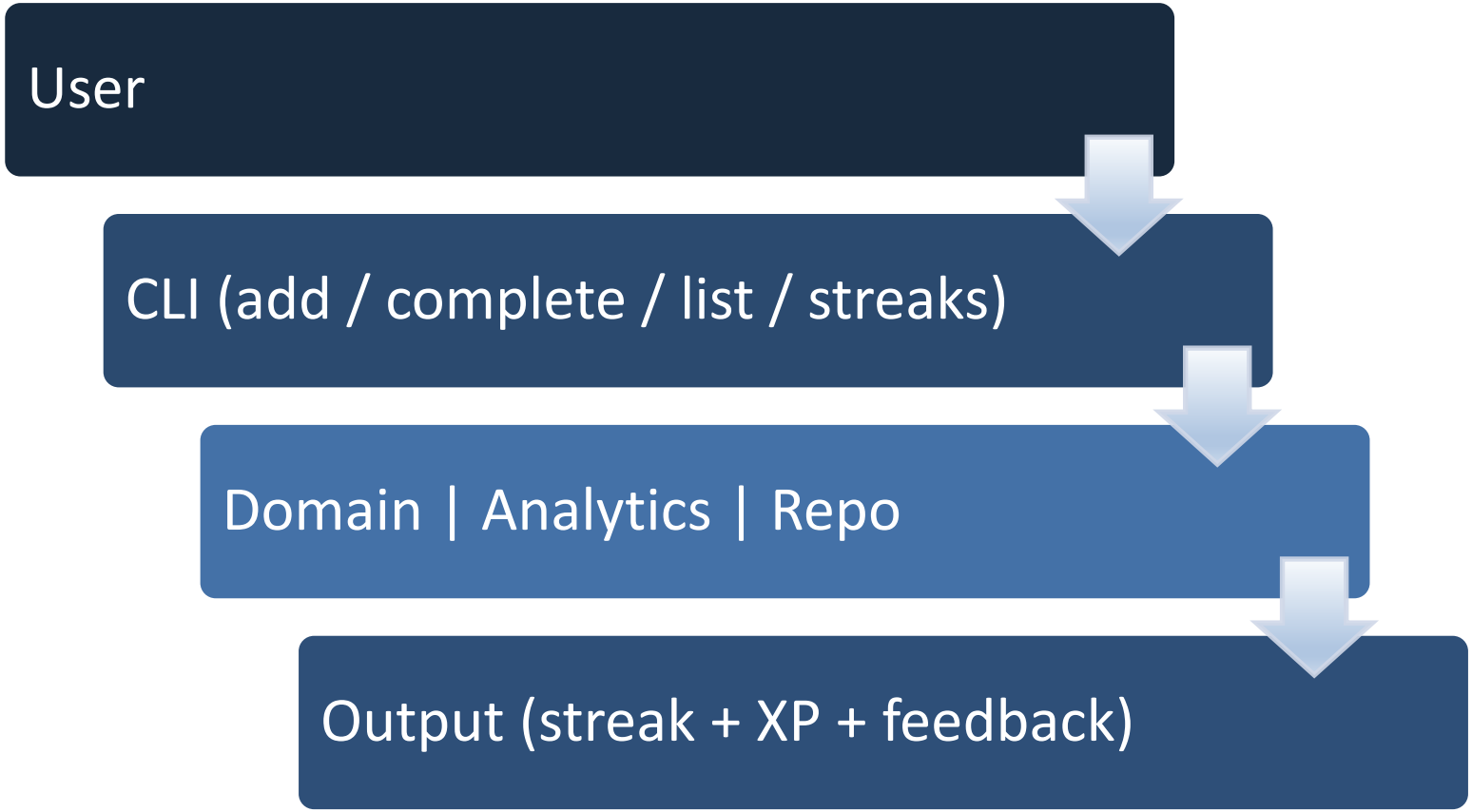
Habit Class & Event Log Structure

- Habit: id, name, periodicity, created_at
- Completion: habit_id, timestamp, optional note
- Event-log concept

Storage Approach & Data Flow

- SQLite repository layer isolates database operations
- Check-off flow:
  - CLI → HabitTracker → Repository (insert completion) → Analytics + UX feedback
- Analytics flow:
  - Repository (load completions) → Functional streak calculation → CLI output

CLI Workflow: Create → Check-off → View



Analytics Functions (FP) – Design & Status

- Pure functional streak calculations
- Daily and weekly streak logic
- Current and longest streak per habit
- Deterministic, side-effect free and testable analytics

Predefined Habits & 4-Week Test Data

- 5 predefined habits (3 daily, 2 weekly)
- Example fixture data covering 4 weeks
- Daily habits: simulated daily completions
- Weekly habits: simulated weekly completions
- Used for testing and demonstration

Testing Strategy & Early Results

- pytest-based unit tests
- domain layer tested
- analytics functions tested
- persistence layer tested
- 7 tests passed

Progress vs. Original Plan

- Architecture implemented as designed
- Layer separation maintained
- Functional analytics completed
- SQLite persistence integrated
- UX enhancements (gamification + sentiment) added

Next Steps Toward Phase 3

- Improve fixture loading process
- Refactor datetime to timezone-aware objects
- Improve documentation (README)
- Prepare codebase for optional UI extension